

DeepSeek Harness

多设备控制与移动能力扩展架构设计

基于单一权威 PC Runtime、Tailscale 私有设备网络与轻量 Mobile Capability Client 的 Human-Agent 多设备工作环境

1 摘要

本方案将 DeepSeek Harness (DSH) 从单机工作站上的 Agent runtime 扩展为多设备个人计算环境。PC 保持唯一权威运行时，承担 LLM 请求、Agent Loop、会话与上下文、插件运行时、绝大多数 Tool 执行和数据密集型计算；手机承担移动交互、内容采集、用户在场性与 Android 原生能力。Tailscale 提供私有设备网络、设备身份与受控连通性，使移动端无需把 DSH 服务暴露到公网即可访问同一 WebUI、同一会话和同一 workspace，并可反向接收 PC Agent 发出的结构化 Tool invocation。

设计不变量：One authoritative runtime • One session truth • One workspace • Many human surfaces • Many capability endpoints。跨设备扩展发生在 presentation 与 capability 层，状态权威和任务编排保持集中。

执行集中 模型调用、上下文管理、Tool 调度和多数计算保留在 PC，延续现有 DSH runtime 的性能路径与定制能力。	能力分布 手机仅执行其物理设备或 Android OS 独有的 capability，例如相机、位置、Intent、通知与传感器。
Surface 复用 移动 App 直接加载 PC 托管的 DSH WebUI，壁纸、音乐、sidebar、session 和后续 UI 插件随 PC 端更新自动同步。	权限下沉 Tool 语义在 PC 解析，最终副作用由 capability 所在设备的权限代理授权并执行；网络可信不替代本地授权。

2 背景与运行时基线

现有 PC 端 DSH 已经形成较深的 session/context governance 定制。其一，基于 leavelet/deepseek-harness 引入有界长会话读取、packed retention、上下文检查与区间选择、历史召回、按模型容量规划压缩和可恢复摘要审阅。其二，基于 MuWinds/dsh-archived-sessions 增加归档会话列出、预览、释放、删除与容量统计。其三，基于 xiaobright/dsh-anchored-standard 引入受控首轮 Tool surface、上下文门控、wire-think 路由、压缩感知阶段提升、session prefab 播种、跨平台 shell path 与稳健指令发现。

这一基线直接改变移动化方案的成本结构。完整 Android Harness 会复制 session state、context pipeline、plugin lifecycle 和升级路径，并要求上述定制长期两端维护。移动端的主要价值来自移动控制、内容采集与设备能力，因此更适合把 Agent runtime 保留在 PC，把手机压缩为 presentation client 与 capability endpoint。

3 目标与边界

目标	设计含义
私有远程控制	手机在 5G、校园网或异地 Wi-Fi 下通过 tailnet 访问 PC DSH；PC 服务无需公网端口、DDNS 或公网反向代理。
单一状态权威	会话、context、archive、workspace、Agent 状态和 Tool result commit 均以 PC runtime 为准，移动端不维护第二份 authoritative state。
移动能力纳入 Tool graph	PC Agent 可以调用手机相机、位置、文件选择、Intent 等 capability；Tool schema、调度和结果归档仍由 PC ToolRuntime 管理。
UI 与插件同步	Android App 使用 WebView/PWA 加载同一 DSH Web application；PC 更新 UI 或插件后，移动端无需同步实现同一 presentation layer。
性能路径保持	LLM streaming、Agent Loop、PC-local Tool 并发和重计算保持原执行路径；跨设备 RTT 只作用于 mobile-native Tool 与移动 presentation。

首版范围排除独立离线移动 Agent、双端 session replication、公开多租户服务以及依赖 root 的系统级控制。独立手机运行能力可以作为未来分支，但不进入当前主架构。

4 总体架构

系统可分为 Control Plane、Compute Plane 和 Capability Plane。Control Plane 接收来自 Desktop、Phone、未来 Voice/Tablet surface 的 Human 指令并统一落入 PC DSH；Compute Plane 在 PC 处理 LLM、Agent Loop、会话与绝大多数 Tool；Capability Plane 允许每个设备注册自身不可替代的本地能力。Tailscale 连接三个平面，同时保持 DSH 自身对 session、Tool 和 permission semantics 的控制。

Control Plane Human → any surface → PC DSH Agent
Compute Plane PC DSH → LLM / session / shell / files / jobs / heavy compute
Capability Plane device endpoint → device-local capability / side effect
Transport Tailscale tailnet → private reachability + device identity

4.1 PC 权威运行时

PC 节点持有 canonical session store、context pipeline、workspace、Agent Loop 和 ToolRuntime。执行路由器位于 ToolRuntime 与 executor 之间：PC-local Tool 直接进入本机 executor；device-local Tool 依据 capability descriptor 路由至指定设备。结果统一回到 PC 后进入 session event/history，使 replay、compression、archive 与 observability 仍围绕一份事件事实工作。

4.2 轻量移动客户端

移动客户端由两部分组成。WebView/PWA 负责 presentation，直接加载 PC 的 DSH Web application；Native Capability Service 负责 Web 无法稳定或安全完成的系统能力，包括 Share Target、相机、位置、Android Intent、文件选择、后台服务与后续通知/传感器能力。两部分共享设备身份与本地权限状态，但不在手机重新实现 LLM adapter、Agent Loop、context manager 或通用 plugin runtime。

5 Tool 执行语义

一个手机本地操作应在 PC 完成 Tool lookup、schema validation、调度、权限需求判定与 execution target 选择。手机只接收结构化调用，不再次解析自然语言。该边界避免形成第二套 Agent runtime，也使 Tool 调用可以与 PC-local Tool 进入同一 task graph。

```
ToolInvocation { call_id, session_id, device_id, capability, args, deadline, approval_policy, idempotency_key }
ToolResult { call_id, status, data | error, device_state, timing, audit_metadata }
```

路由粒度落在 capability，而非整个 Agent。产品层可以提供“优先设备”或“默认执行节点”偏好，但执行系统依据 Tool descriptor 选择目标。一个任务可以依次调用 phone.camera.capture、PC 视觉/文本处理、pc.web.fetch、pc.git.* 与 pc.files.write，无需切换全局“手机模式”或“PC 模式”。

5.1 文件与内容迁移

普通手机文件处理采用“上传一次、PC 本地继续执行”的策略。Share Target 或文件选择器把 PDF、图片、URL、文本等 artifact 写入 PC workspace 或 session attachment 后，解析、检索、转换、代码生成与归档均在 PC 完成。该策略减少跨设备重复读取和移动端 I/O 依赖，也使现有文件 Tool、索引与上下文机制直接复用。

5.2 动态 Capability Manifest

手机上线后向 PC 注册动态 capability manifest。每项能力携带 schema version、availability、OS permission、approval requirement、foreground requirement、concurrency class 与可选资源约束。权限撤销、设备锁定、App 退到后台或硬件不可用时，manifest 发生变化，PC 随之更新可用 Tool surface。

Capability	典型状态	执行约束
phone.camera.capture	available / user-presence-required	相机通常按设备串行；必要时拉起前台 UI。
phone.location.get	granted / denied / foreground-only	精度、后台可用性与 OS permission 绑定。
phone.file.pick	available	通过系统选择器授予单次文档访问，随后上传 PC。
phone.intent.open	available / approval-required	产生外部 UI 或跨 App 副作用时记录审批。
phone.notification.*	special-access / denied	进入后续阶段；读与写/回复采用不同风险等级。

6 移动交互模型

手机承担三类 surface。第一类为 Control Surface：继续同一 session、发送 prompt、处理 approval、观察 job、interrupt/resume。第二类为 Capture Surface：通过系统 Share Sheet、文件选择、相机、语音等把手机当前内容送入 DSH。第三类为 Presentation Surface：复用 PC DSH WebUI，包括 sidebar、壁纸、音乐、session inspector 与后续工作台插件。

关键体验：移动端看到的是同一运行时的另一个 surface。PC 端修改 WebUI、插件或工作台结构后，移动端通过刷新同一 Web application 获得更新；APK 仅在新增 Android 原生 capability 时需要升级。

Share Target 可以提供轻量目标选择，例如 session、workspace 与动作类型。收到的 artifact 进入统一 ingest pipeline：识别 MIME/URL → 写入 workspace 或临时 attachment → 绑定 session → 触发 Agent task。该通路覆盖网页、GitHub 链接、论文 PDF、相册图片、选中文本和未来语音片段。

7 网络与权限模型

Tailscale 用作 device fabric。Android 8 及以上设备可运行官方客户端 [10]。tailscale serve 可以把 PC 上仅监听 localhost 的 DSH Web 服务私有暴露到 tailnet，公开到 Internet 的路径属于 Funnel，二者安全边界不同 [11]。连接优先采用设备间

UDP 直连，直连不可用时回退到 Peer Relay 或 DERP；三类连接均使用 WireGuard 端到端加密，性能差异主要来自路径长度 [12]。

网络可达性只构成第一层授权。完整权限链分为四层：

1. **Tailnet policy**。通过显式 grants 约束 phone → PC Web/API 与 PC → phone capability RPC 的方向、目标和端口；Grants 支持 deny-by-default 的细粒度授权模型 [13]。部署时应覆盖初始宽松策略，避免 tailnet 内无关节点自动获得 DSH 控制面。
2. **设备与应用身份**。PC DSH 与 Mobile Client 完成 pairing，绑定稳定 device identity；RPC 请求携带 device/session/call identity，并由双方拒绝未知 peer。
3. **Android OS permission**。未获得 camera/location 等系统权限时，对应 capability 不注册或标记 unavailable。
4. **DSH capability grant 与 invocation approval**。低风险能力可按 once/session/always 授权；发送消息、修改外部状态、录音、通知回复等高副作用动作保留逐次确认和审计。

8 性能与生命周期

该架构避免改变 PC 上的 LLM serving path 和主 Agent runtime。模型请求、streaming parser、context management 与 PC-local Tool 继续原位执行；Tailscale 新增的 RTT 主要作用于移动 UI event stream 和 phone-native Tool RPC。由此，模型侧 TTFT 与 tokens/s 仍主要取决于原 provider/网络/PC 运行时，移动化不会引入第二层模型代理或 Android 端 runtime serialization。

Tailscale 直连通常提供最低延迟与最高吞吐，受 NAT/防火墙限制时 relay 会增加时延 [12]。系统应保持 WebSocket/SSE 长连接，避免为每个 token 或 Tool event 重建连接；大文件通过一次性上传迁移到 PC；phone-local Tool 按 I/O 操作处理，不承担持续大吞吐数据平面。相机、麦克风、屏幕等独占资源需要每设备并发约束，PC 侧 Tool scheduler 可以继续并发调度互不冲突的本地任务。

8.1 失效与恢复

故障	处理机制
手机离线 / 网络切换	heartbeat/lease 使 capability 快速转为 unavailable；重连后重新注册 manifest，不重放未知副作用。
RPC 超时或重复投递	call_id + idempotency_key + deadline；副作用 Tool 记录执行状态并拒绝重复提交。
权限运行时撤销	Mobile Client 更新 manifest；正在执行的调用返回结构化 permission error，PC 记录到原 Tool result。
Android App 被系统回收	需要持续任务的 capability 使用受控 foreground/background service；presentation lifecycle 与 capability service 分离。
PC Web client 重连	session 与 job authority 在 PC runtime，WebView/浏览器断开不终止任务；重新连接后恢复 event/state projection。

9 既有方案横向对比

现有社区项目分别验证了远程移动 UI、原生 Android client、本地 Android runtime、PRoot compatibility layer 和 ADB device control。其技术目标并不相同，因此比较依据是本方案的目标适配度，而非通用优劣。

方案	执行权威	移动 UI	手机原生 capability	状态/维护特征	对本目标的适配
本方案	PC DSH	WebUI + native shell	一等能力；RPC + permission broker	一套 session/context/plugin runtime；设备只注册 capability	同时覆盖远程控制、PC 强环境和手机 OS 能力
saya-ch/dsh-mobile [4]	PC DSH	成熟；同一 Web origin/WebView	主要为移动访问与可定制 UI	不修改 DSH；同一 session/workspace	最接近 presentation/remote-access 基线
Hakunm/dsh-android-app [5]	PC DSH	原生 Android client	重点在聊天、审批、文件、模型管理	远程 API 维持同一 workspace/session/files	native client 与远程管理参考
fengnanrui/DSH-Android [6]	手机本地	原生 Android	可原生实现	复制 session/Agent/plugin 语义；独立维护	适合 standalone/offline；不符合单一权威目标
WSK-build/DSHBox [7]	手机本地 DSH	本机 WebUI	Android 能力仍需桥接	PRoot Debian + Node + DSH	兼容 upstream runtime；companion 场景成本偏高
ZSeven-W/dsh-android [8]	PC DSH	DSH 内设备 panel	ADB 深度控制 Android	20 个 Agent tools；ADB serial identity	证明 PC Agent → Android device 可行
0xcaff/codex-web [9]	Host Codex	Desktop UI → Browser	无手机 OS bridge	Electron IPC → WebSocket shim；最小 patch	runtime/client lifecycle separation 参考

9.1 比较结论

dsh-mobile 已验证“同一 PC DSH + 移动 Web surface”的产品路径；dsh-android-app 验证原生移动控制；dsh-android 验证 PC Agent 可以把 Android 设备纳入 Tool graph；codex-web 验证 desktop Agent runtime 与 client presentation 可通

过薄协议层解耦。当前设计把这些方向收敛为一套统一边界：PC 保留 state/compute authority，手机只增加 surface 与 capability authority。与本地 Android Harness 相比，主要代价是依赖 PC 在线和 tailnet 连通；主要收益是避免 runtime duplication，并保留 PC 的完整 shell/files/git/process 环境与既有 DSH 定制。

10 接口与组件设计

10.1 PC 端 DSH Plugin

- **DeviceRegistry**：维护 device identity、在线状态、manifest version、capability descriptors 与 heartbeat lease。
- **ExecutionRouter**：依据 Tool descriptor、device availability、policy 与 resource class 选择 PC executor 或 remote endpoint。
- **MobileRpcTransport**：在 tailnet 上维持认证的双向 RPC/stream，支持 cancellation、deadline、reconnect 和 backpressure。
- **CapabilityToolProjection**：把动态 manifest 投影为 Agent 可见 Tool；不可用 capability 从 surface 移除或显式标记。
- **AuditLog**：记录 invocation、approval、device、result、timing 与错误类别，并与 session event 建立关联。

10.2 Android Client

- **Web shell**：加载 tailnet 内 DSH WebUI，处理安全 origin、登录/配对、深链与文件上传。
- **Capability Service**：接收结构化 invocation、维护 manifest、调用 Android adapter、返回结构化结果。
- **Permission Broker**：统一 OS permission、应用级 grant、Human approval 和风险级别。
- **Share Receiver**：接收 ACTION_SEND/ACTION_SEND_MULTIPLE 等输入并上传至指定 session/workspace。
- **Lifecycle Service**：仅为确有持续需求的能力保持 foreground/background execution，避免把 UI Activity 当作运行时。

11 实施路径

Phase 0 — Network baseline	PC 保持 DSH 仅监听 localhost，通过 Tailscale Serve 或 tailnet IP 暴露 WebUI；验证 session continuity、streaming、job reconnect 与移动网络访问。
Phase 1 — Thin client	Android shell：WebView/PWA、pairing、Share Target、文件/URL ingest、基础通知。UI 仍由 PC Web application 提供。
Phase 2 — Capability RPC	实现 registry、manifest、RPC envelope、permission broker 和首批能力：device.info、file.pick/upload、camera.capture、location.get、intent.open。
Phase 3 — DSH integration	capability 投影为原生 Tool/service，加入 execution router、Tool-level target selection、audit、timeout/cancel/idempotency。
Phase 4 — Ambient control	扩展语音、通知、后台任务、传感器、BLE；加入风险分级与 user-presence policy。
Phase 5 — Multi-device	registry 泛化到 tablet/server/home machine；GPU、桌面 shell 与手机传感器进入同一 capability graph。

12 验证指标

维度	验证问题	建议测量
响应性能	移动化是否显著增加 token/event 可见延迟？	PC local 与 phone-over-tailnet 的 TTFT presentation delta、event RTT；直连与 relay 分开统计。
Tool latency	phone-native Tool 的额外网络成本是否可接受？	camera/location/intent invocation → result wall-clock，拆分 network/approval/Android API。
状态一致性	断网、重连、重复 RPC 是否产生重复副作用或 session divergence？	fault injection：drop/reorder/retry/reconnect，检查 call_id 与 event log。
权限正确性	网络可达时是否仍能拒绝未授权 capability？	grant matrix、OS permission revoke、锁屏、后台状态与 unknown device 测试。
维护成本	PC runtime 更新后移动端是否保持可用？	WebUI/plugin 更新回归；仅 native API contract 变化时升级 APK。

13 风险与开放问题

首要风险来自 Android lifecycle 与权限边界：后台 execution、通知访问、录音、屏幕与跨 App 操作需要分别评估系统限制和发行渠道政策。第二类风险来自远程副作用的幂等性与审计；普通 read Tool 可以透明重试，外部写操作需要明确 idempotency 与 Human approval。第三类风险来自 WebUI 与 native bridge 的版本契约，应以小而稳定的 capability

protocol 隔离 DSH Web 前端变化。第四类风险来自网络连接类型差异，直连与 DERP relay 的性能需要实测并纳入可观测性，而不把 overlay network 延迟归因到模型推理。

14 架构决策

ARCHITECTURE DECISION

采用“PC 单一权威 DSH Runtime + Tailscale 私有设备网络 + 轻量 Mobile Capability Client”作为主路线。

PC 继续承担模型调用、Agent Loop、session/context、workspace、plugin runtime 和多数 Tool；移动端复用 PC WebUI，并通过独立 capability service 暴露 Android-native operations。Tool routing 按 capability/设备细分，Human 可以从任意 surface 发起任务。该决策优先保留现有 DSH 性能路径与深度定制，降低双运行时维护和状态同步复杂度，同时把手机从远程控制器提升为可授权的物理设备 capability endpoint。

15 参考资料

1. [leavelet/deepseek-harness](https://github.com/leavelet/deepseek-harness). <https://github.com/leavelet/deepseek-harness>
2. [MuWinds/dsh-archived-sessions](https://github.com/MuWinds/dsh-archived-sessions). <https://github.com/MuWinds/dsh-archived-sessions>
3. [xiaobright/dsh-anchored-standard](https://github.com/xiaobright/dsh-anchored-standard). <https://github.com/xiaobright/dsh-anchored-standard>
4. [saya-ch/dsh-mobile](https://github.com/saya-ch/dsh-mobile). <https://github.com/saya-ch/dsh-mobile>
5. [Hakunm/dsh-android-app](https://github.com/Hakunm/dsh-android-app). <https://github.com/Hakunm/dsh-android-app>
6. [fengnanrui/DSH-Android](https://github.com/fengnanrui/DSH-Android). <https://github.com/fengnanrui/DSH-Android>
7. [WSK-build/DSHBox](https://github.com/WSK-build/DSHBox). <https://github.com/WSK-build/DSHBox>
8. [ZSeven-W/dsh-android](https://github.com/ZSeven-W/dsh-android). <https://github.com/ZSeven-W/dsh-android>
9. [Oxcaff/codex-web](https://github.com/Oxcaff/codex-web). <https://github.com/Oxcaff/codex-web>
10. [Tailscale — Install on Android](https://tailscale.com/docs/install/android). <https://tailscale.com/docs/install/android>
11. [Tailscale Serve](https://tailscale.com/docs/features/tailscale-serve). <https://tailscale.com/docs/features/tailscale-serve>
12. [Tailscale — Connection types](https://tailscale.com/docs/reference/connection-types). <https://tailscale.com/docs/reference/connection-types>
13. [Tailscale — Grants](https://tailscale.com/docs/features/access-control/grants). <https://tailscale.com/docs/features/access-control/grants>

资料状态：2026-09-09。社区仓库处于快速演进阶段，版本、插件 ID 与实现细节应在实现阶段重新核验。